

TypeScript Mental Obstacles & Solutions

The confusions and their resolutions, with an example for every rule — for the G2i code-review assessment

Compiled for Jeanne Shih (Prompterminal)

October 2026

Preface

The assessment is code review in TypeScript, not coding, so every rule here is about reading code and spotting what is wrong. Each section pairs numbered **mental obstacles** (the confusion, stated flat) with numbered **solutions & rules** (one-sentence resolutions), each followed by a one-line example marked × (bug) or ✓ (correct). A **Python** → **TypeScript** block closes each language section; a short vocabulary note (what `{}` and `[]` are) opens Section I. Memorize Sections VII and VIII first; read I–VI to make VII make sense.

I. Foundations: the JavaScript runtime model

Mental obstacles

1. Variables were imagined as boxes holding values, so `b = a` on an object looked like a copy.
2. `==` and `===` looked interchangeable, and `=` inside an `if` looked like a typo rather than a bug.
3. Truthiness was assumed to match Python, so `[]` and `{}` were read as falsy.
4. `null` and `undefined` looked like the same thing with two names.
5. Which array methods mutate and which return new arrays was not visible from the call site.
6. `const` was read as “cannot change” when it means “cannot rebind”.

Solutions & rules

0. The literal tells you the structure: `{}` is an object (the dict), `[]` is an array (the list), `new Map()` a map, `new Set()` a set; `o.x` / `o["x"]` read a key, `undefined` when missing.
`const o = { x: 1 }; o.x // 1 — const a = [1, 2]; a[5] // undefined`
1. A JS variable is a label bound to a value; objects and arrays are shared by reference, primitives are copied by value.
`let a = 1, b = a; b++; // a is 1 — const o = {x:1}, p = o; p.x = 2; // o.x is 2`
2. `b = a` on an object or array creates two labels on ONE object; `b.push(4)` is visible through `a`.
`const a = [1,2]; const b = a; b.push(3); a // [1,2,3]`
3. `[...a]`, `a.slice()`, `{...o}`, `Array.from(a)` make shallow copies; nested objects are still shared until `structuredClone(o)`.
`const c = {...o}; c.inner.x = 9; // x o.inner.x also 9 — structuredClone(o) // ✓ independent`
4. `===` compares value and type with no coercion; `==` coerces; expect `===` and flag `==`.
`0 == "" // true × — 0 === "" // false ✓ — null == undefined // true`
5. `=` is assignment and returns the assigned value, so `if (x = 5)` is always true and `if (o.status = "done")` overwrites `status`.
`if (o.status = "paid") {} // × assigns, always truthy — if (o.status === "paid") {} // ✓`

6. Falsy values are exactly `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`; everything else is truthy.
`if ([]) console.log("runs"); // runs` — `if ("0") console.log("runs"); // runs`
7. `if (arr)` never checks emptiness; `if (arr.length)` or `arr.length === 0` does.
`if (!items) return; // x never true for []` — `if (items.length === 0) return; // ✓`
8. `undefined` is “never assigned”; `null` is “deliberately empty”; `typeof null` is `"object"`.
`const o: {a?: number} = {}; o.a // undefined` — `function f() {} f() // undefined`
9. `a ?? b` falls back only on `null/undefined`; `a || b` falls back on any falsy value.
`const n = count || 10; // x 0 becomes 10` — `const n = count ?? 10; // ✓ 0 stays 0`
10. `o?.a?.b` short-circuits to `undefined` instead of throwing when `o` or `o.a` is nullish.
`user.address.city // x throws if address missing` — `user.address?.city // ✓ undefined`
11. `const` prevents reassignment of the label, not mutation of the object.
`const a = []; a.push(1); // ✓ legal` — `a = [2]; // x TypeError`
12. `let` is block-scoped; `var` is function-scoped and hoisted, and is a smell in modern TS.
`for (var i = 0; i < 3; i++) {} i // 3, leaked` — `for (let i...) {} i // ReferenceError`
13. Mutating array methods: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, `fill`; `sort()` sorts IN PLACE and returns the same array.
`const sorted = arr.sort(); // x arr is also sorted now` — `const sorted = [...arr].sort(); // ✓`
14. Non-mutating: `map`, `filter`, `slice`, `concat`, `flat`, `flatMap`, `reduce`, `find`, `findIndex`, `some`, `every`, `includes`, `indexOf`, `join`, `toSorted`, `toReversed`.
`arr.filter(x => x > 0); // x result discarded, arr unchanged` — `const pos = arr.filter(x => x > 0); // ✓`
15. `arr.sort()` with no comparator sorts as STRINGS; numeric sort needs `(a, b) => a - b`.
`[10, 9, 1].sort() // x [1, 10, 9]` — `[10, 9, 1].sort((a, b) => a - b) // ✓ [1, 9, 10]`
16. Strings are immutable: every string method returns a new string, `s[0] = "x"` silently does nothing.
`s.toUpperCase(); // x s unchanged` — `s = s.toUpperCase(); // ✓`
17. Floating point: `0.1 + 0.2 !== 0.3`; money in floats is a bug, integer cents is the fix.
`19.99 * 100 // 1998.9999999999998 x` — `Math.round(19.99 * 100) // 1999 ✓`
18. `typeof x` returns a string; `Array.isArray(x)` is the only reliable array check.
`typeof [] // "object"` — `Array.isArray([]) // true ✓`
19. `NaN !== NaN`; use `Number.isNaN(x)`; `parseInt("12px")` is `12`, `Number("12px")` is `NaN`.
`if (x === NaN) // x never true` — `if (Number.isNaN(x)) // ✓`
20. Object keys are always strings (or symbols); `Map` keeps key types.
`const o = {}; o[1] = "a"; o["1"] // "a", same key` — `new Map().set(1, "a").get("1") // undefined`
21. `for...of` iterates values; `for...in` iterates keys as strings; `for...in` on an array is a bug.
`for (const i in [10, 20]) typeof i // "string" x` — `for (const v of [10, 20]) // 10, 20 ✓`
22. `JSON.parse` throws on bad input; `JSON.stringify` drops `undefined` and functions.
`JSON.parse(body) // x unguarded` — `try { JSON.parse(body) } catch { return 400 } // ✓`

Reading notes: nine things that confuse on first read

A. “Undefined” is the value; “falsy” is how it behaves. They answer two different questions and are both true at once.

```
const o: { a?: number } = {}; // type: object that MAY have key a (a number);
value: empty object
o.a // undefined <- the VALUE (key was never set)
o.a === undefined // true <- testing the value
if (o.a) { ... } // does not run <- falsy BEHAVIOR
!o.a // true <- falsy behavior
```

“Falsy” is a label for how a value is treated where a boolean is expected (`if`, `!`, `&&`, `||`). Exactly eight values carry it: `false`, `0`, `-0`, `0n`, `""`, `null`, `undefined`, `NaN`. `undefined` is one of them, the way `0` is. Python parallel: `d.get("a")` is `None`, and `None` is also falsy. Review consequence: `if (!o.a)` cannot tell “missing” from “set to 0”; the precise test for missing is `o.a === undefined` or `o.a == null`.

B. Truthiness: where Python instincts are wrong.

Value	Python	TypeScript
<code>[]</code>	falsy	truthy
<code>{}</code>	falsy	truthy
<code>set()</code> / <code>new Set()</code>	falsy	truthy
<code>"0"</code> , <code>"false"</code>	truthy	truthy
<code>0</code> , <code>""</code> , <code>None</code> / <code>null</code> / <code>undefined</code>	falsy	falsy

So `if not items:` has no direct TS form: `if (!items)` never fires for `[]`; write `if (items.length === 0)`. And query strings arrive as strings, so `req.query.enabled` is `"0"`, which is truthy; convert before testing.

C. `===` vs `==`: *strict vs coerced*. `===` asks “same type AND same value?” with no conversion. `==` first converts both sides to a common type (coercion), then compares, with surprising results.

Expression	<code>===</code>	<code>==</code>
<code>5</code> vs <code>"5"</code>	false	true
<code>0</code> vs <code>""</code>	false	true
<code>0</code> vs <code>"0"</code>	false	true
<code>""</code> vs <code>"0"</code>	false	false
<code>null</code> vs <code>undefined</code>	false	true
<code>[]</code> vs <code>false</code>	false	true

Python’s `==` is already strict about type (`5 == "5"` is `False`), so Python `==` $\approx$ TS `===`; TS `==` has no Python equivalent. In review: expect `===/!==` everywhere; a `==` is a low-severity but real finding, because it lets `"5"` pass as `5` or `""` as `0`. The one idiomatic exception is `x == null`, which deliberately catches both `null` and `undefined`. Separately, a single `=` is assignment, never comparison (rule 5).

D. `.sort((a, b) => a - b)`: *what the comparator means*. With no argument, `.sort()` converts every element to a string and sorts alphabetically (“lexicographic”), so `[10, 9, 1].sort()` gives `[1, 10, 9]` because `"10"` starts with `"1"`, which comes before `"9"`. To sort numerically you pass a **comparator**: a function of two elements that returns a number telling sort which goes first.

```

(a, b) => a - b    // read: "given two elements a and b, return a minus b"
// negative -> a goes before b; positive -> b goes before a; zero -> leave as
is

[10, 9, 1].sort((a, b) => a - b)           // [1, 9, 10] ascending
[10, 9, 1].sort((a, b) => b - a)           // [10, 9, 1] descending
users.sort((a, b) => a.score - b.score)    // objects, by a field
names.sort((a, b) => a.localeCompare(b))  // strings, alphabetical

```

Python's `sorted(xs, key=lambda x: x.score)` says *what to sort by*; TS has no `key=`, so you say *how to compare two items*. Two planted bugs follow: `ids.sort()` on numbers with no comparator (wrong order), and a comparator that returns a boolean, `(a, b) => a.score > b.score`, which is not negative/zero/positive and gives unreliable order; it must be `b.score - a.score`. And `.sort()` mutates in place (rule 13), so use `[...xs].sort(...)` to keep the original.

E. `for...in` vs `for...of`: *keys vs values*. Python has one loop and what you get depends on what you iterate (`d`, `d.values()`, `d.items()`). TS has two keywords that look alike: **of** = the values of, **in** = the keys in (always as strings).

Python	TypeScript
<code>for k, v in d.items():</code>	<code>for (const [k, v] of Object.entries(d))</code>
<code>for k in d:</code>	<code>for (const k of Object.keys(d))</code>
<code>for v in d.values():</code>	<code>for (const v of Object.values(d))</code>
<code>for x in xs:</code>	<code>for (const x of xs)</code>

`Object.entries(d)` turns `{ a: 1, b: 2 }` into `[["a", 1], ["b", 2]]`; `for...of` walks that array and `const [k, v]` destructures each pair, exactly like Python's `k, v`. On an array, `for (const x of xs)` gives elements; `for (const i in xs)` gives the indices `"0"`, `"1"` as **strings**, so `xs[i + 1]` becomes `xs["01"]` → `undefined`. `for...in` on an array is a planted bug; flag it. A `Map` needs no `Object.entries`: `for (const [k, v] of map)` works directly and keys keep their real type.

F. `??` vs `||`: “if missing” vs “if falsy”. Both read “use `a`, otherwise `b`”; they differ in what counts as otherwise. `||` falls back when `a` is any of the eight falsy values; `??` falls back only when `a` is `null` or `undefined`.

a	a 10	a ?? 10
<code>undefined</code>	10	10
<code>null</code>	10	10
<code>0</code>	10	0
<code>""</code>	10	<code>""</code>
<code>false</code>	10	<code>false</code>
<code>5</code>	5	5

Python's `a or b` behaves like `||` (`0 or 10` is `10`); Python has no `??`, the nearest is `a if a is not None else b`. The bug: `const n = count || 10` is meant as “default when not given”, but a caller passing `count = 0` gets 10. `count ?? 10` keeps the 0. Rule of thumb: if the left side can legitimately be `0`, `""` or `false` (counts, page numbers, IDs from 0, flags, `size`, `offset`, `timeoutMs`), `||` is wrong and `??` is right. Exercise 2 plants exactly this with `timeoutMs || DEFAULT`. Cousins: `x ??= 5` assigns only if nullish, `x ||= 5` if falsy, `x &&=`

`f(x)` if truthy. In a plain condition (`if (a || b)`) `||` is ordinary boolean OR; the trap is only when `||` picks a value.

G. `const` locks the label, not the contents. `a.push(1)` does not create a new `a`; it reaches into the existing array `a` points to and adds `1` inside it. `a = [2]` tries to point the label at a different array, and that is what `const` forbids.

```
const a = [];           // a --> [ ]           label a, pointing at an empty array
a.push(1);              // a --> [ 1 ]         SAME array, contents changed (mutation)
✓
a = [2];                // a --> [ 2 ] x      label re-pointed at a NEW array
(reassignment) TypeError
```

	What changes	Allowed with <code>const</code> ?
Mutation: <code>a.push(1)</code> , <code>a[0] = 9</code> , <code>a.sort()</code> , <code>o.x = 2</code>	the contents of the thing <code>a</code> points to	yes
Reassignment: <code>a = [2]</code> , <code>a = null</code> , <code>a = a.filter(...)</code>	which thing the label <code>a</code> points to	no

Python has no `const`, but the two operations are the in-place vs rebinding distinction from the Python notes (`a.append(1)` vs `a = [2]`); TS `const` just forbids the second. Review consequence: `const` on an array or object does not make the data safe from change; `const orders = ...; orders.sort(...)` still reorders the caller's array (Exercise 1). For primitives (`const n = 1`) there is nothing inside to mutate, so `const` is fully fixed; the distinction only shows up with objects, arrays, `Map`, `Set` and class instances. Freezing contents is separate: `Object.freeze(a)` at runtime (shallow) or `readonly number[]` as a type.

H. The arrow `=>` is a lambda; non-mutating methods return a NEW array. `x => x > 0` is TS for `lambda x: x > 0`: parameter on the left, returned expression on the right.

```
x => x > 0               // one parameter, parentheses optional (lambda x: x > 0)
(x) => x > 0             // same
(a, b) => a - b          // two parameters need parentheses (the sort
comparator)
() => 42                 // no parameters
x => { x * 2 }           // x braces = block body, no return -> undefined (rule
III.2)
x => { return x * 2 }    // ✓
```

The arrows in rule 14 are callbacks: you hand the function to the method and it is called once per element. `arr.filter(x => x > 0)` reads as “go through `arr`, call `x => x > 0` on each element, keep the ones where it returns `true`”; Python `[x for x in arr if x > 0]`. The point of the rule: `filter`, `map`, `slice`, `concat`, `flat`, `toSorted`, `toReversed` leave `arr` alone and hand back a **new** array as the return value; if nothing catches that value it is thrown away, exactly like Python's `sorted(xs)` on its own line.

```
arr.filter(x => x > 0);           // x new array created and dropped; arr
unchanged; line does nothing
const pos = arr.filter(x => x > 0); // ✓ caught in pos; arr still unchanged
```

Returns a new value, must be assigned	Changes the array in place, return value is a trap
<code>map</code> , <code>filter</code> , <code>slice</code> , <code>concat</code> , <code>flat</code> , <code>flatMap</code> , <code>toSorted</code> , <code>toReversed</code> (new array)	<code>push</code> , <code>pop</code> , <code>shift</code> , <code>unshift</code> , <code>splice</code> , <code>sort</code> , <code>reverse</code> , <code>fill</code>
<code>reduce</code> , <code>find</code> , <code>findIndex</code> , <code>some</code> , <code>every</code> , <code>includes</code> , <code>indexOf</code> , <code>join</code> (a value)	

Two planted-bug shapes: a non-mutating call on its own line with nothing assigned (`arr.map(...)`, `s.toUpperCase()`), which does nothing; and a mutating call assigned as if it were a copy (`const sorted = arr.sort()`), where both names are the same, now-sorted array (rule 13).

I. The arrow is not an `if`; it returns. `x => x > 0` has no `if` in it: `=>` reads “returns”, so it is a function that returns the boolean `x > 0` (`f(5)` is `true`, `f(-3)` is `false`). The deciding happens inside the method, not the arrow: `filter` wraps your condition in its own loop and `if`.

```

arr.filter(x => x > 0)           // what filter does internally, roughly:
const out = [];
for (const x of arr) {
  if (x > 0) out.push(x);       // <- the if is filter's; the condition x > 0 is
yours
}

```

The arrow can return anything, not only booleans; what the method does with the return value depends on the method:

Call	The arrow returns	What the method does with it
<code>arr.filter(x => x > 0)</code>	boolean	keeps <code>x</code> if <code>true</code>
<code>arr.map(x => x * 2)</code>	a new value	puts it in the new array
<code>arr.find(x => x.id === id)</code>	boolean	returns the first <code>x</code> where <code>true</code>
<code>arr.sort((a, b) => a - b)</code>	a number	uses the sign to order <code>a</code> and <code>b</code>
<code>arr.some(x => x < 0)</code>	boolean	returns <code>true</code> if any was <code>true</code>

An `if/else` inside an arrow is the ternary, Python’s `a if c else b`: `x => x > 0 ? "pos" : "neg"`; or a block body with real `if` statements and an explicit `return`: `x => { if (x > 0) return "pos"; return "neg"; }`.

Python → TypeScript

1. Same label-not-box model; copy idioms differ: `a[:]` / `a.copy()` → `[...a]` / `{...o}` / `structuredClone(o)`.
2. Python `==` (value) / `is` (identity) → TS `==` (coerced) / `===` (strict); the rule flips: Python prefer `==`, TS always `===`.
`x == None` → `x === undefined`
3. `=` in a condition is a `SyntaxError` in Python; in TS it compiles and silently assigns.
`if (s = "a")` // compiles `x`
4. Python falsy includes `[]`, `{}`; TS empty arrays and objects are TRUTHY.
`if not lst:` → `if (lst.length === 0)`
5. Python has one `None`; TS has `null` AND `undefined`.
`d.get("k")` # `None` → `obj["k"]` // `undefined`

6. Python raises `KeyError/IndexError`; TS returns `undefined` and keeps running.
`xs[99] # IndexError → xs[99] // undefined`
7. Python `sorted()` new / `.sort()` returns `None`; TS `.sort()` mutates AND returns the same array.
`b = sorted(a) → const b = [...a].sort()`
8. Python `sorted([10,9,1])` numeric; TS `[10,9,1].sort()` lexicographic.
`→ .sort((a, b) => a - b)`
9. Python `int/float`; TS one `number`.
`7 // 2 # 3 → Math.floor(7 / 2) // 3`
10. Python `for x in d` keys; TS `for...in` keys-as-strings, `for...of` values.
`for k, v in d.items() → for (const [k, v] of Object.entries(d))`
11. Strings immutable in both; Python throws on `s[0] = "x"`, TS silently ignores.
12. Python has no `const`; TS `const` enforces never-rebind for the label only.

II. Reading TypeScript: types and syntax

Mental obstacles

1. Type annotations looked like extra noise that changed what the code did at runtime.
2. `interface` and `type` looked like two unrelated features.
3. Generics (`Promise<T>`, `Map<K, V>`) looked like a separate language.
4. `as`, `!` and `any` looked like fixes when they are the places where the type system was switched off.
5. The difference between a type error and a runtime bug was not visible.

Solutions & rules

1. Types are erased at compile time; nothing after a `:` in a declaration runs.
`const n: number = "5" as any; // compiles, n is "5" at runtime`
2. `x: string` declares its type; `(a: number, b: string): boolean =>` declares params then return.
`function f(id: string, n = 0): Promise<User>`
3. Primitives: `string`, `number`, `boolean`, `null`, `undefined`, `bigint`, `symbol`; `void` = returns nothing useful; `never` = never returns.
`function fail(msg: string): never { throw new Error(msg) }`
4. `string[]` and `Array<string>` are the same; `[string, number]` is a tuple.
`const pair: [string, number] = ["a", 1];`
5. `{ id: number; name?: string }` is an object type; `?` makes the property `string | undefined`.
`const u: {name?: string} = {}; u.name.length // x type error, possibly undefined`
6. `interface` and `type` both name a shape; treat them as the same for review.
`interface A { x: number } = type A = { x: number }`
7. `A | B` union; `A & B` intersection; `"a" | "b"` literal union.
`type Status = "paid" | "pending"; const s: Status = "done"; // x type error`
8. A generic is a type parameter: `Promise<number>`, `Map<string, number[]>`, `Record<string, X>`, `Partial<T>`.
`const m: Map<string, number[]> = new Map(); m.get("k") // number[] | undefined`

9. `function f<T>(x: T): T` means “whatever goes in comes out”; read `T` as a placeholder.
`function first<T>(xs: T[]): T | undefined { return xs[0] }`
10. `any` turns off checking; every `any` is a place a wrong-shape bug can hide.
`const data: any = await res.json(); data.userName // x typo not caught`
11. `unknown` is the safe `any`; must be narrowed before use.
`const d: unknown = JSON.parse(s); if (typeof d === "object" && d && "id" in d) {...}
// ✓`
12. `x as string` is a cast that does no runtime check.
`const key = req.headers["x-api-key"] as string; // x really string | string[] |
undefined`
13. `x!` asserts non-null and does nothing at runtime.
`const u = users.find(u => u.id === id!); u.name // x crashes if not found`
14. `as const` freezes literals; `readonly` is compile-time only.
`const ROLES = ["admin", "user"] as const; // type: readonly ["admin", "user"]`
15. Narrowing: `typeof`, `instanceof`, `in`, `!= null`; inside the block the type is narrower.
`function len(x: string | string[]) { return typeof x === "string" ? x.length :
x.join().length }`
16. `(x: Shape): x is Circle` is a type guard; a wrong body lies to the compiler.
`const isCircle = (s: Shape): s is Circle => "radius" in s;`
17. `enum Color { Red, Green }` compiles to a real numeric object; string literal unions are preferred.
`enum E { A, B } E.A === 0 // true`
18. `satisfies` checks without widening; `declare` says “exists elsewhere”; neither runs.
`const cfg = { port: 80 } satisfies Config;`
19. `import { x }` named; `import x` default; one `export default` per file.
`import express from "express"; import { Router } from "express";`
20. `?.` and `??` are runtime JS; `?` after a property and `!` after an expression are compile-time TS.
`user?.name ?? "anon" // runs — user!.name // only compiler affected`
21. A type can be right and the code still wrong; types catch shape errors, not logic errors.
`const total: number = price - discount; // ✓ types, x if spec said price + tax`
22. Classify each suspicious line: runtime bug, type hole (`any/as/!`), or style.
`res.json() as User // type hole: missing await hides as a cast`

Python → TypeScript

1. Python hints are ignored; TS types are erased too, but the compiler REJECTS mismatches, so a built `.ts` file has had every annotation checked.
2. `Optional[str]` → `string | undefined` (or `name?: string`).
3. `Union[A, B]` → `A | B`; `Literal["a","b"]` → `"a" | "b"`; `TypedDict/dataclass` → `interface`.
4. `List[int]` → `number[]`; `Dict[str,int]` → `Record<string, number>` or `Map<string, number>`;
`Tuple[str,int]` → `[string, number]`.
5. `Any` → `any`; TS adds `unknown`, which Python lacks.
6. `typing.cast(T, x)` → `x as T`; TS adds `x!` with no Python equivalent.
7. `isinstance(x, str)` → `typeof x === "string"`; `isinstance(x, MyClass)` → `x instanceof MyClass`.

8. Python `Enum` members are objects; TS `enum` compiles to numbers (`Color.Red === 0`).
9. No generics syntax to read past in Python bodies; TS `<T>` is a placeholder to skip.
10. Python duck-types at runtime; TS structurally types at compile time: right fields = the interface, no `implements` needed.
11. `from m import x` → `import { x } from "./m"`; `import m` → `import * as m`; TS adds `export default`.

III. Functions, arrays, objects

Mental obstacles

1. Arrow functions, `function` declarations and methods looked like three different things.
2. `map/filter/reduce` chains read as prose but the data flow was not traced.
3. `forEach` looked like a `for` loop that could `return` or `break`.
4. Destructuring and spread looked like pattern-matching magic.
5. Default parameters and the Python mutable-default trap were assumed to work the same way.
6. `this` was assumed to mean the enclosing object.

Solutions & rules

1. Arrow, `function`, and class method all define functions; the arrow captures surrounding `this`.
`const f = (x: number) => x * 2; function g(x: number) { return x * 2 }`
2. An arrow with a brace body needs an explicit `return`.
`xs.map(x => { x * 2 }) // x [undefined, ...] — xs.map(x => x * 2) // ✓`
3. An arrow returning an object literal needs parentheses.
`() => { a: 1 } // x undefined — () => ({ a: 1 }) // ✓`
4. No `return` → `undefined`.
`function f() { if (x) return 1 } // returns undefined when !x`
5. Read a chain as a pipeline; trace one sample element through every step.
`orders.filter(o => o.paid).map(o => o.amount).reduce((a, b) => a + b, 0)`
6. `map` same length; `filter` subset; `find` first or `undefined`; `some/every` booleans; `reduce` accumulator.
`[1,2,3].find(x => x > 5) // undefined — [].every(x => x > 0) // true`
7. `reduce` without initial value THROWS on empty.
`[].reduce((a, b) => a + b) // x TypeError — [].reduce((a, b) => a + b, 0) // ✓ 0`
8. `forEach` ignores returns, cannot `break`, does not await.
`xs.forEach(x => { if (x < 0) return; }) // only skips that item`
9. `for...of` supports `break`, `continue`, `return`, `await`.
`for (const x of xs) { if (x < 0) break; } // ✓`
10. `includes/indexOf` `O(n)`; `set.has` `O(1)`.
`const seen = new Set(ids); seen.has(id) // ✓`
11. `find` → element; `findIndex/indexOf` → `-1` when absent; `if (arr.indexOf(x))` is wrong.
`if (arr.indexOf(x)) {} // x false for index 0, true for -1 — if (arr.includes(x)) // ✓`

12. `splice` removes in place; `slice` copies; `arr[-1]` is undefined.
`arr[-1] // x undefined — arr.at(-1) // ✓ last`
13. `{ a, b } = obj` pulls fields; `{ a: x }` renames; `{ a = 5 }` defaults; `[x, , z]` skips.
`const { id, name = "anon" } = user; const [first, , third] = arr;`
14. `...` spreads in a literal, collects in a parameter list.
`const all = [...a, ...b]; function f(first: number, ...rest: number[]) {}`
15. In `{ ...defaults, ...input }`, later keys win.
`{ ...input, ...DEFAULTS } // x defaults override input — { ...DEFAULTS, ...input } // ✓`
16. Default parameters are evaluated on EVERY call; no mutable-default trap.
`function f(xs: number[] = []) { xs.push(1); return xs } f(); f() // [1], [1] ✓`
17. A default applies only when the argument is `undefined`.
`function f(n = 10) {} f(null) // n is null — f(0) // n is 0`
18. A closure captures the variable; `var` in a loop shares one `i`, `let` gives each iteration its own.
`for (var i = 0; i < 3; i++) setTimeout(() => log(i)) // x 3,3,3 — let i // ✓ 0,1,2`
19. `this` depends on the call; passing `obj.method` as a callback loses it.
`btn.on("click", this.handle) // x this undefined — btn.on("click", () => this.handle()) // ✓`
20. `Object.keys/values/entries` return arrays.
`for (const [k, v] of Object.entries(o)) {}`
21. `k in obj` checks a key; `obj[k] !== undefined` mis-treats a key set to `undefined`.
`"a" in { a: undefined } // true — ({ a: undefined }).a !== undefined // false`
22. `f?.()` optional call; `arr?.[0]` optional index.
`cb?.(result); const first = list?.[0];`
23. Template literals interpolate; user input into URL/SQL/shell/HTML unescaped is a security finding.
`fetch(`/users/${id}`) → fetch(`/users/${encodeURIComponent(id)}`)`
24. `private/public` are compile-time; `#x` is runtime-private.
`class A { private x = 1; #y = 2 } (a as any).x // 1, readable`
25. `static` lives on the class.
`class A { static make() {} } A.make() // ✓ — new A().make() // x`

Python → TypeScript

1. `lambda x: x * 2` → `(x) => x * 2`; a TS arrow with `{}` needs `return`.
2. `def` with no `return` → `None`; TS → `undefined`, plus the arrow-brace trap.
3. `list(map(f, xs))` → `xs.map(f)`, eager, chainable.
4. `[f(x) for x in xs if p(x)]` → `xs.filter(p).map(f)`; reading order flips.
5. `functools.reduce(f, xs, init)` → `xs.reduce(f, init)`.
6. `for x in xs: ... break` → `for (const x of xs) { break }`, never `forEach`.
7. `xs[-1]` → `xs.at(-1)`.
8. `xs[1:3]` → `xs.slice(1, 3)`; `del xs[1:3]` → `xs.splice(1, 2)`.
9. `x in xs` → `xs.includes(x)`; `k in d` → `k in obj` (on arrays `in` checks indices `x`).
10. `a, b = pair` → `[a, b] = pair`; `{**d1, **d2}` → `{ ...d1, ...d2 }`; `*args` → `...args`.

11. `def f(xs=[])` shared $\times$; `function f(xs = [])` fresh $\checkmark$.
12. `self` explicit and bound; `this` implicit and call-dependent.
13. Python closures share the loop variable; TS `let` gives per-iteration binding.
14. `f"{x}"` $\rightarrow$ ``${x}``.
15. `__init__` $\rightarrow$ `constructor`; `@staticmethod` $\rightarrow$ `static`; `_private` $\rightarrow$ `private` or `#field`.

IV. Async: promises, await, fetch, timers

Mental obstacles

1. `async/await` looked like it made code run in order, so a missing `await` was invisible.
2. A `Promise<T>` and a `T` looked the same, so `const data = fetchData()` was read as having the data.
3. A `try/catch` around an un-awaited promise was assumed to catch its rejection.
4. Sequential awaits were not recognized when they could have been parallel.
5. Node being single-threaded was taken to mean “no race conditions”.
6. `setTimeout` and `setInterval` looked harmless.

Solutions & rules

1. An `async` function ALWAYS returns a `Promise`.
`async function f() { return 1 } f() // Promise<number>, not 1`
2. `await p` unwraps; without it the variable holds a `Promise`.
`const u = getUser(id); u.name //  $\times$  undefined — const u = await getUser(id); //  $\checkmark$`
3. A missing `await` is the second most common planted bug.
`const data = res.json() as User; //  $\times$  — const data = (await res.json()) as User; //  $\checkmark$`
4. `try { f() } catch {}` does NOT catch an async rejection; `return f()` inside `try` escapes too.
`try { save() } catch (e) {} //  $\times$  — try { await save() } catch (e) {} //  $\checkmark$`
5. A fire-and-forget async call with no `.catch` is a finding.
`notify(user); //  $\times$  unhandled rejection — notify(user).catch(log); //  $\checkmark$`
6. Sequential awaits on independent work is an efficiency finding.
`const a = await A(); const b = await B(); // slow — const [a, b] = await Promise.all([A(), B()]);`
7. `Promise.all` rejects on first failure; `allSettled` waits for all; `race` first to settle; `any` first to fulfil.
`await Promise.allSettled(tasks) // [{status, value|reason}, ...]`
8. `await` inside `forEach` does nothing useful.
`ids.forEach(async id => out.push(await get(id))); return out; //  $\times$  [] — await Promise.all(ids.map(get)) //  $\checkmark$`
9. `await` in `for...of` is sequential by design.
`for (const id of ids) await get(id); // one at a time, intentional or not`
10. Read $\rightarrow$ await $\rightarrow$ write on shared state is a race.
`const n = counter.get(k); await db.save(); counter.set(k, n + 1); //  $\times$  lost updates`
11. Cache check-then-fill is the canonical race; store the `Promise`.
`if (!cache[k]) cache[k] = await load(k); //  $\times$  — cache[k] ??= load(k); return await cache[k]; //  $\checkmark$`

12. `fetch` resolves on ANY HTTP status; check `res.ok`.
`const d = await res.json(); // x parses 404 body — if (!res.ok) throw new Error(String(res.status)); // ✓`
13. `res.json()` returns a Promise, can throw, is typed `any`.
`const d: unknown = await res.json(); // ✓ then validate`
14. `fetch` has no default timeout.
`fetch(url, { signal: AbortSignal.timeout(5000) }) // ✓`
15. Wrapping a promise-returning call in `new Promise` is redundant.
`new Promise(r => fetch(u).then(r)) // x — fetch(u) // ✓`
16. An uncleared `setInterval` leaks and keeps Node alive unless `.unref()`.
`setInterval(sweep, 60_000); // x — setInterval(sweep, 60_000).unref(); // ✓`
17. `Date.now()` is a number; `Date` objects compare by identity.
`d1 === d2 // x false for equal times — d1.getTime() === d2.getTime() // ✓`
18. Hard-coded `Date.now()` makes logic untestable; inject a clock.
`function limiter(now: () => number = Date.now) {} // ✓`
19. Order: sync → microtasks (`await`, `.then`) → macrotasks (`setTimeout`).
`setTimeout(() => log("T"), 0); Promise.resolve().then(() => log("P")); // P then T`
20. `async` arrow in `.map` gives Promises; wrap in `Promise.all`.
`const users = ids.map(async id => get(id)); // x Promise[] — await Promise.all(...) // ✓`
21. `catch (e)` is `unknown`; narrow before `e.message`.
`catch (e) { log(e.message) } // x — catch (e) { if (e instanceof Error) log(e.message) } // ✓`
22. Swallowing errors is a finding unless the requirement said to.
`catch (e) {} // x silent — catch (e) { throw new Error("load failed", { cause: e }) } // ✓`

Python → TypeScript

1. Python is sync by default, `asyncio` opt-in; Node is async by default, every I/O returns a Promise.
2. `async def/await` ↔ `async function/await`.
3. Forgotten `await`: Python warns and never runs the coroutine; TS RUNS it and hands you a Promise — silent.
4. `asyncio.gather(a(), b())` → `Promise.all([a(), b()])`; `return_exceptions=True` → `allSettled`.
5. `await asyncio.sleep(1)` → `await new Promise(r => setTimeout(r, 1000))`; no `time.sleep`.
6. `try/except` around `await` is the same; the un-awaited-inside-try trap is TS-only.
7. `requests.get(...).raise_for_status()` → `if (!res.ok) throw`.
8. `response.json()` sync → `await res.json()`.
9. Threads race in Python; TS races at every `await`.
10. `threading.Timer` rare → `setTimeout/setInterval` everywhere; uncleared interval keeps Node alive.
11. `except Exception as e` typed → `catch (e)` is `unknown`.
12. `datetime.now()` object → `Date.now()` ms number / `new Date()` object.

V. Nulls, errors, and input handling

Mental obstacles

1. A function was judged on its happy path only.
2. Type annotations on a parameter were taken as proof the value had that shape at runtime.
3. Throwing and returning an error value looked equivalent.
4. HTTP status codes were not checked against the requirement.

Solutions & rules

1. Types are a promise about internal callers; external data arrives as `any/unknown` and must be validated at the boundary.

```
const body = req.body as CreateUser; // x trust — const body =  
CreateUserSchema.parse(req.body); // ✓
```
2. For every parameter ask: `undefined`, `null`, empty, zero, negative, NaN, huge, wrong type, extra fields.

```
summarize([]) // what is returned? — page(-1, 0) // rejected?
```
3. `req.headers["x"]` is `string | string[] | undefined`; `req.params.id` is a `string`; `req.body` is `any`.

```
const id: number = req.params.id; // x type error, it is a string —  
Number(req.params.id)
```
4. Reject a missing required header/key BEFORE use; `undefined` as a map key merges all callers.

```
const key = req.headers["x-api-key"]; hits.get(key) // x all missing keys share one  
bucket
```
5. `Number(input)` on garbage is `NaN`, which passes every `</>` as false.

```
const size = Number(q.size); if (size > 100) reject // x NaN slips through — if  
(!Number.isInteger(size) || size < 1 || size > 100) // ✓
```
6. `throw` aborts; returning `null/-1` relies on every caller checking.

```
function find(id) { return users.find(...) ?? null } find(id).name // x caller forgot
```
7. `throw "bad"` loses the stack; `throw new Error("bad")`; custom classes allow `instanceof`.

```
class NotFound extends Error {} if (e instanceof NotFound) res.status(404)
```
8. A re-throw must keep the cause.

```
catch (e) { throw new Error("failed") } // x cause lost — throw new Error("failed", {  
cause: e }) // ✓
```
9. `finally` always runs; cleanup belongs there.

```
try { await work() } finally { clearTimeout(t); conn.release() }
```
10. Express 4 does not catch errors thrown in async handlers.

```
app.get("/", async (req, res) => { throw new Error() }) // x hangs — .catch(next) or  
express-async wrapper ✓
```
11. `res.json()` without `return` keeps executing and can double-send.

```
if (!ok) res.status(401).json(e); doWork(); // x — if (!ok) return  
res.status(401).json(e); // ✓
```
12. Status codes: 200/201/204, 400 bad input, 401 no creds, 403 not allowed, 404, 409, 429, 500; spec headers are checklist items.

```
missing token → 401, wrong org → 403, bad page → 400
```

13. `Retry-After` is positive integer seconds.
`Math.ceil(ms / 1000) // x can be 0 — Math.max(1, Math.ceil(ms / 1000)) // ✓`
14. Validate ranges, not only presence.
`if (size) // x allows -1 and 1e9 — if (size >= 1 && size <= 100) // ✓`
15. User input in URL → `encodeURIComponent`; SQL → parameters; shell → no shell; HTML → escape.
`db.query(`... WHERE id = '${id}'`) × — db.query("... WHERE id = $1", [id]) ✓`
16. Secrets in source, logging tokens, stack traces to clients are security findings.
`const KEY = "sk_live..."; console.log({ token }) // x`
17. `if (!user)` also fires on `0`, `""`, `false`; for IDs that can be `0` use `== null`.
`if (!userId) return 400; // x rejects id 0 — if (userId == null) // ✓`
18. `arr[i]` past the end is `undefined`, never an exception.
`for (let i = 0; i <= arr.length; i++) arr[i].x // x crashes on last iteration, one past`
19. `obj[key]` with user keys can hit `__proto__`/`constructor`; use `Map`.
`handlers["constructor"].push(h) // x it is a function — new Map() // ✓`
20. Empty-collection behavior: `Math.max(...[])` is `-Infinity`, `[] .reduce(f)` throws, `[] .every` true, `[] .some` false.
`Math.max(...scores) // -Infinity on []`

Python → TypeScript

1. Python raises on bad access; TS returns `undefined` and crashes one step later.
2. `raise ValueError("m")` → `throw new Error("m")`; TS allows `throw "m"` (smell).
3. `except ValueError:` → `catch (e) { if (e instanceof ValueError) ... }`.
4. `try/except/finally` → `try/catch/finally`; no `else` clause.
5. `int("12px")` raises → `Number("12px")` is `NaN`, `parseInt` is `12`.
6. `is None` → `x == null` (the one idiomatic `==`).
7. `if not x` on a list is real → `if (!x)` on an array is not.
8. Flask/FastAPI parse and validate → Express gives raw `any`.
9. pydantic model → zod schema; a TS `interface` does NOT validate.
10. Flask `return` ends the request → Express `res.json()` does not end the function.
11. f-string into SQL × in both; parameters in both.
12. No prototype chain in Python → plain-object key hazard is TS-only.

VI. Collections, Map/Set, complexity

Mental obstacles

1. Plain objects and `Map` looked interchangeable as dictionaries.
2. Counter/defaultdict/deque were looked for by name.
3. Nested loops were not read as $O(n^2)$ when a lookup structure would make them $O(n)$.
4. A cache that only grows was not seen as a leak.

Solutions & rules

1. `Map<K, V>` keeps order, any key type, `.size`, no prototype.

```
const m = new Map<string, number>(); m.set("a", 1); for (const [k, v] of m) {}
```
2. `Set<T>` dedupes and tests membership.

```
[...new Set([1, 1, 2])] // [1, 2]
```
3. `map.get(k)` is `undefined` when absent.

```
map.get(k).push(x) // x crash — (map.get(k) ?? []).push(x) or if (!map.has(k))
map.set(k, []) // ✓
```
4. Counter → `map.set(k, (map.get(k) ?? 0) + 1)`; `defaultdict(list)` → `?? []`; deque → array; heapq → sort or library.

```
for (const w of words) count.set(w, (count.get(w) ?? 0) + 1);
```
5. `Record<string, T>` is a plain-object map; fine for trusted keys, smell for user keys.

```
const byName: Record<string, User> = {}; byName[req.query.name] // x user key
```
6. Array costs: index/push/pop $O(1)$; shift/unshift/splice/indexOf/includes/find $O(n)$; `includes` in a loop $O(n \cdot m)$.

```
a.filter(x => b.includes(x)) //  $O(n \cdot m)$  — const bs = new Set(b); a.filter(x =>
bs.has(x)) //  $O(n+m)$ 
```
7. `Map/Set` get/set/has/delete $O(1)$ average.
8. Sliding-window log: `filter` $O(\text{window})$; `shift` while expired is cheaper; fixed buckets $O(1)$.

```
while (stamps.length && now - stamps[0] >= WINDOW) stamps.shift();
```
9. `filter` inside a handler allocates per call; fine unless throughput is the requirement.
10. A cache with no eviction, no TTL sweep and user keys grows without bound.

```
cache.set(req.query.q, result) // x unbounded — cap size (LRU) or validate keys ✓
```
11. A sweep every N minutes still allows growth between sweeps.

```
setInterval(sweep, 10 * 60_000) // 10 minutes of unique junk keys
```
12. Computing a pruned copy and discarding it on the reject path diverges state; minor.

```
const recent = ts.filter(...); if (recent.length >= L) return 429; // x recent not
stored
```
13. `WeakMap/WeakRef` keys are GC-able; a sign of leak awareness.

```
const meta = new WeakMap<object, Meta>();
```
14. `structuredClone` deep-copies; `JSON.parse(JSON.stringify(o))` drops `undefined`, `Date`, `Map`, `Set`.

```
JSON.parse(JSON.stringify({ d: new Date() })).d // string x
```
15. `arr.length = 0` empties in place; `arr = []` rebinds.

```
const a = [1]; const b = a; a.length = 0; b // [] (shared)
```
16. `Object.freeze` is shallow; `readonly` is compile-time only.

```
const o = Object.freeze({ inner: { x: 1 } }); o.inner.x = 2 // ✓ allowed, shallow
```
17. Comparators return numbers; a boolean comparator is a bug.

```
.sort((a, b) => a.score > b.score) // x — .sort((a, b) => b.score - a.score) // ✓
desc
```
18. `Object.groupBy/Map.groupBy` exist; manual reduce is fine.

```
Map.groupBy(docs, d => d.tag)
```
19. Big-O ranking: $O(1) \rightarrow O(\log n) \rightarrow O(n) \rightarrow O(n \log n) \rightarrow O(n^2) \rightarrow O(2^n)$; call out only when it matters.
20. Multi-process deployment makes every in-memory `Map` per-process; a design finding, stated once.

```
const hits = new Map(); // per instance; with 3 replicas the limit is 3x
```

Python → TypeScript

1. One `dict` → plain object (string keys, prototype) or `Map` (any key, no prototype).
2. `set` → `Set`; no `|/∧` operators: `new Set([...a].filter(x => b.has(x)))`.
3. `d[k]` raises / `d.get(k)` None → both `map.get(k)` and `obj[k]` return `undefined`.
4. `Counter/defaultdict/deque/heapq` → no built-ins; see rule 4 idioms.
5. `len(xs)` → `xs.length` (array/string) or `map.size/set.size`.
6. `list.pop(0)` ↔ `arr.shift()`, both $O(n)$; `append` ↔ `push`, $O(1)$.
7. `copy.deepcopy` → `structuredClone`; `list(xs)/dict(d)` → `[...xs]/{...d}`, shallow in both.
8. `sorted(xs, key=lambda x: x.s, reverse=True)` → `[...xs].sort((a, b) => b.s - a.s)`; no `key=`.
9. `.sort()` returns `None` vs returns the array; both mutate.
10. dict insertion order → plain objects put integer-like keys first (`{2: a, 1: b}` iterates `1, 2`); `Map` is pure insertion order.
11. `itertools.groupby` needs sorted input → `Map.groupBy` does not.
12. Same Big-O per structure; the syntax hides which structure you have.

VII. Code-review bug checklist (the planted-bug catalogue)

Mental obstacles

1. Reading code once top to bottom felt like reviewing it; nothing was checked against the requirement.
2. Every issue found was listed with equal weight.
3. “What went well” was skipped or filled with “it compiles”.
4. Findings were described as symptoms instead of consequences.

Solutions & rules

1. Pass 1 requirement clauses; Pass 2 what each block does; Pass 3 checklist against each clause and block.
Req: “at most 100 per 60 s, 429 with Retry-After” → clauses: (a) limit 100, (b) rolling 60 s, (c) 429, (d) header.
2. For every clause, point at the line that satisfies it or write “not implemented”; a missing clause is the top finding.
Clause (d): no `res.set("Retry-After")` anywhere → finding #1.
3. Operator check: `=` in conditions, `==`, boundaries with the limit value itself.
`if (count > LIMIT) // test 100: passes x, test 101: blocked → should be >=`
4. Off-by-one: loop bounds, slice ends, `length - 1`, `ceil/floor` on exact integers.
`for (let i = 0; i <= n; i++) // n+1 iterations — attempt < 3 // 2 attempts`
5. Await check: every async call has `await/return/.then`; every async `forEach` is a bug.
`const u = res.json(); sleep(200); ids.forEach(async ...) // three findings`
6. Null check: every property access on a maybe-undefined; every `!` and `as`.
`users.find(...).name — map.get(k).push — headers.auth as string`
7. Input check: missing, empty, zero, negative, NaN, wrong type, too large, user-controlled key.
`page=0, size=-1, size=abc, orgId=__proto__`

8. Error check: `res.ok`; `JSON.parse` in try; swallowing catch; thrown strings; send without return.
`catch (e) {} — throw "x" — res.status(401).json(); next()`
9. State check: read-await-write race; cache without eviction; interval without clear/unref; per-process state.
`const hit = cache.get(k); await fetch(); cache.set(k, ...)`
10. Security check: unescaped input into URL/SQL/shell/HTML; auth missing or after work; secrets in code;
`eval`.
``WHERE org='${orgId}'` — req.query.orgId ?? claims.orgId`
11. Efficiency check: `includes`/`find` inside a loop; sequential awaits; rank below correctness unless perf is the requirement.
`for (const a of A) if (B.includes(a)) // O(n·m)`
12. Type-hole check: `any`, `as X`, `x!`, `@ts-ignore`, required field the data can omit.
`const cfg = JSON.parse(raw) as Config; // every field unverified`
13. Return check: arrow braces without `return`; declared `number` but a path returns `undefined`.
`(m, v) => { v > m ? v : m } // undefined`
14. Mutation check: `sort`/`splice` on caller data; parameter mutated; shared default object.
`orders.sort(...) // caller's array reordered`
15. Dead-code and lie check: comment $\neq$ code; unreachable branch; unused var; misleading name.
`// retries 3 times` above `attempt < 3` — `const recent = ...` // then never stored
16. Rank: (a) violates requirement/crashes, (b) wrong on edge, (c) security, (d) leak/race, (e) design limit, (f) efficiency, (g) style.
 $1. \equiv \text{in status filter (a)} \cdot 2. \text{ no } \text{res.ok} \text{ (b)} \cdot 3. \text{ token logged (c)} \cdot 4. \text{ no eviction (d)} \cdot 5. \text{ per-process (e)} \cdot 6. \text{ includes loop (f)}.$
17. Always find real positives; “it compiles” does not count.
“Sliding-window log is the right structure for a rolling window; cleanup job shows leak awareness.”
18. State each finding as line $\rightarrow$ consequence $\rightarrow$ fix.
“Line 14 uses `>` so the 101st request passes; use `>=`.”
19. One sentence of consequence per finding; three well-explained beat ten listed.
20. A design limitation is stated once with “acceptable if X; otherwise needs Y”.
“Per-process Map: acceptable single-instance; otherwise Redis.”
21. Verdict tree: clause missing/core wrong $\rightarrow$ reject or request changes (low); edge/security/leak only $\rightarrow$ request changes (medium); efficiency/style only $\rightarrow$ approve with comments (high).
22. Confidence = how sure the code does what the requirement says; 1 = core fails, 5 = every clause and edge traced; never default to 3.

VIII. Test-day reference

Mental obstacles

1. Too much material; one page was needed to read before opening the test.
2. Talking through code on camera without a script produced rambling.
3. Using AI on the test felt like cheating, so it was avoided or hidden.
4. Time was not budgeted.

Solutions & rules

1. Time plan (~90 min): 0:00–0:10 requirement clauses; 0:10–0:25 first read; 0:25–0:55 checklist pass; 0:55–1:10 four fields; 1:10–1:30 Loom. Up to 5–6 h total is fine, one sitting, submit within 24 h.
2. The four fields, written BEFORE recording: **Confidence (1–5)** + one sentence; **What went well** (2–4); **What went wrong** (ranked, line → consequence → fix); **Overall opinion** (approve / request changes / reject + deciding reason).
Confidence: 2 — the limit is off by one and missing keys share a bucket.
3. Confidence scale: 1 core fails; 2 serious bug on realistic input; 3 happy path ok, edges wrong; 4 correct with nits; 5 fully traced.
4. Loom script (5–8 min, camera on): (a) restate requirement 30 s; (b) walk code 2 min; (c) findings ranked, point at line, consequence, 3 min; (d) rating and verdict 1 min.
“The requirement is 100 per rolling minute with a 429 and Retry-After. The code keeps a timestamp array per key... At line 14...”
5. Say what you checked and found fine.
“I tested the boundary: request 100 is allowed, 101 is blocked, so the limit is correct.”
6. AI is allowed as a helper; name every use in one sentence.
“I looked up `Record<string, number[]>`; it is a map from key to timestamp array.”
7. Every finding must be defensible on camera without notes.
8. Unknown construct: look it up, one line in notes, continue.
??= — assigns only if nullish.
9. Stuck past 5 min: write “possible issue, not confirmed: ...” and move on.
“Possible race on cache miss; not confirmed without seeing concurrency requirements.”
10. Keyword → first suspicion: rate limit → boundary + missing-key bucket; cache/TTL → race + eviction; fetch → `res.ok`, `await`, timeout; retry → attempt count, max; debounce → timer clear, `this`; pagination → size range, offset; auth → check order, secret, id source; money → floats, `=` in filter; parse/JSON → unguarded parse, `any`.
11. Edge inputs: `undefined`, `null`, `""`, `[]`, `{}`, `0`, `-1`, `NaN`, limit, limit + 1, one element, duplicates, huge n, two concurrent callers.
12. Checklist order: requirement → `=/==` → boundary → `await` → null → input → errors → state → security → efficiency → style.
13. Before submitting: every finding has line, consequence, fix; ranked; positives real; verdict matches top finding; AI uses named; camera on.
14. Night before: Loom installed, 30-second test watched back, paper copy of VII.16 and VIII.12, sleep.